SAT ENCODING CHO BÀI TOÁN SUDOKU VÀ ĐÁNH GIÁ THỰC NGHIỆM SO SÁNH GIẢI SUDOKU $N \times N$ VỚI CÁC PHƯƠNG PHÁP CHÍNH XÁC (CP, ILP/MIP)

Nguyễn Minh Sơn - 24021614[a,*]

[a] *Các vấn đề hiện đại về KHMT, Khoa công nghệ thông tin, Trường đại học Công nghệ*

ARTICLE INFO

Keywords:
Sudoku Solver
SAT Encoding
AMO Constraints
Constraint Programming
Integer Linear Programming
PySAT
Glucose3
OR-Tools
Gurobi
CPLEX

Tóm tắt nội dung

Bài toán Sudoku tổng quát trên ma trận kích thước $N \times N$ ($4 \times 4, 9 \times 9, 16 \times 16$) là bài toán thuộc lớp NP-complete kinh điển, mang tính chất đại diện tiêu biểu cho các bài toán thỏa mãn ràng buộc (CSP) và giải đố logic. Báo cáo bài tập lớn này tập trung tìm hiểu, thiết kế và đánh giá thực nghiệm các kỹ thuật mã hóa mệnh đề logic (SAT Encoding) cho các ràng buộc số lượng (Cardinality Constraints), đặc biệt là ràng buộc At-Most-One (AMO), At-Least-One (ALO) và Exactly-One (EO). Nội dung thực hiện triển khai thực nghiệm 5 kỹ thuật mã hóa AMO: *Pairwise (Naive)*, *Binary (Bitwise)*, *Commander (Klieber & Kwon, 2007)*, *2-Product (Jingchao Chen, 2010)*, và *Sequential Counter (Carsten Sinz, 2005)*. Hiệu năng của SAT Solver (*Glucose3* thông qua thư viện PySAT) được đặt trong mối tương quan so sánh đối chứng toàn diện với các công cụ giải chính xác hàng đầu thế giới thuộc hai trường phái Constraint Programming (CP) và Integer Linear Programming (ILP/MIP): *Google OR-Tools CP-SAT*, *IBM CPLEX CP Optimizer*, *Gurobi Optimizer (MIP)*, *IBM CPLEX MIP*, và *PuLP (CBC)*. Thực nghiệm được tiến hành trên tập dữ liệu chuẩn hóa gồm 299 câu đố với đa dạng kích thước ($4 \times 4, 9 \times 9, 16 \times 16$) và 4 cấp độ khó (*Easy, Medium, Hard, Expert*) thu thập từ các kho dữ liệu quốc tế như Peter Norvig Benchmark, Sudoku Exchange Puzzle Bank. Kết quả thực nghiệm cho thấy quy luật sinh biến phụ, số lượng mệnh đề CNF, tính hiệu quả của cơ chế unit propagation và lợi thế về tốc độ của phương pháp SAT cho bài toán Sudoku so với các bộ giải tối ưu hóa truyền thống (CP, ILP/MIP).

1. Giới thiệu (Introduction)

Sudoku là bài toán điền giá trị vào bảng sao cho mỗi hàng, cột và khối con chứa các giá trị không lặp. Với bảng chuẩn 9×9, miền giá trị là 1 đến 9 và mỗi khối có kích thước 3×3. Bài toán có thể mở rộng sang bảng $N \times N$, với $N = k^2$ đối với khối vuông $k \times k$. Đây là một ví dụ phù hợp để khảo sát mô hình hóa bài toán thỏa mãn ràng buộc; Norvig trình bày cách giải bằng lan truyền ràng buộc và tìm kiếm Norvig (2006).

Bài toán quyết định Sudoku tổng quát, trong đó kích thước bảng không cố định mà là một phần của đầu vào, là NP-complete Yato and Seta (2003). Phát biểu này không áp dụng riêng cho một kích thước cố định như 9×9; các bảng 4×4, 9×9 và 16×16 trong báo cáo là những trường hợp thực nghiệm.

SAT (Boolean Satisfiability Problem) xác định sự tồn tại của phép gán làm đúng công thức Boole. Các bộ giải hiện đại như Glucose sử dụng CDCL (Conflict-Driven Clause Learning) Audemard and Simon (2009). Khi áp dụng SAT cho Sudoku, bước quan trọng là chuyển các ràng buộc sang dạng chuẩn hội CNF (Conjunctive Normal Form). Báo cáo tập trung vào ALO (ít nhất một biến đúng), AMO (nhiều nhất một biến đúng) và EO (đúng một biến đúng).

Đồng thời, bên cạnh tiếp cận bằng SAT, bài toán Sudoku còn có thể được mô hình hóa và giải quyết bằng các phương pháp toán học chính xác khác như:

- **Constraint Programming (CP)**: Khai thác các ràng buộc toàn cục như `AllDifferent` thông qua lan truyền miền giá trị cục bộ (domain filtering), tiêu biểu là Google OR-Tools CP-SAT Perron and Furnon (2024) và IBM ILOG CPLEX CP Optimizer IBM Corporation (2024).
- **Integer Linear Programming (ILP / MIP)**: Sử dụng quy hoạch tuyến tính nguyên với các biến nhị phân $\{0, 1\}$, giải quyết bằng phương pháp nhánh cận (Branch-and-Bound) kết hợp vết cắt phẳng (Cutting Planes), tiêu biểu là Gurobi Optimizer Gurobi Optimization, LLC (2024), IBM CPLEX MIP, và CBC Solver.

*Sinh viên.
24021614@vnu.edu.vn (N.M.S.-. 24021614)
ORCID(s):

Nghiên cứu liên quan và phạm vi đóng góp

Các mã hóa ràng buộc đã được nghiên cứu trong các công trình về mã hóa mệnh đề Frisch et al. (2008), Sequential Counter Sinz (2005), Commander Klieber and Kwon (2007) và 2-Product Chen (2010). PySAT cung cấp công cụ để xây dựng và thử nghiệm các mô hình SAT Ignatiev et al. (2018). Báo cáo sử dụng các phương pháp đã có, không đề xuất mã hóa hay thuật toán giải mới. Đóng góp của bài tập lớn là tích hợp các phương pháp trong một hệ thống, đối chiếu quy mô CNF với công thức lý thuyết và khảo sát chi phí thực thi trên cùng đầu vào.

Mục tiêu của bài tập lớn là:

1. Xây dựng hệ thống giải Sudoku tích hợp năm kỹ thuật mã hóa AMO, phân biệt phần tự cài đặt và phần sử dụng thư viện.
2. So sánh các cấu hình SAT dùng những mã hóa khác nhau với các bộ giải CP và ILP/MIP trên cùng đầu vào.
3. Phân tích quy mô mô hình, thời gian thực thi và tính đúng đắn của nghiệm; nêu rõ phạm vi và giới hạn thực nghiệm.

2. Mô hình hóa bài toán Sudoku bằng SAT (SAT Formulation)

Xét bảng Sudoku kích thước $N \times N$ với các khối con kích thước $k_r \times k_c$ ($N = k_r \times k_c$). Mỗi ô trong bảng được định vị bởi tọa độ (r, c) với $0 \le r, c < N$. Các giá trị điền vào nhận giá trị $d \in \{1, 2, \dots, N\}$.

2.1. Biểu diễn biến mệnh đề

Ta định nghĩa biến Boole có ba chỉ số $x_{r,c,d}$:

$$x_{r,c,d} = \begin{cases} 1 & \text{nếu ô } (r,c) \text{ nhận chữ số } d \\ 0 & \text{ngược lại} \end{cases} \tag{1}$$

Mô hình sử dụng N^3 biến Boole cơ sở, mỗi biến tương ứng với một bộ (r, c, d). Để đưa vào bộ giải SAT, các biến được đánh số từ 1 đến N^3 theo ánh xạ:

$$\mathrm{ID}(r, c, d) = N^2 r + Nc + d, \tag{2}$$

với $0 \le r, c < N$ và $1 \le d \le N$. Các biến phụ của encoding AMO được cấp ID lớn hơn N^3.

2.2. Hệ thống ràng buộc của Sudoku

Một cấu hình Sudoku hợp lệ phải thỏa mãn đồng thời 5 nhóm ràng buộc logic sau:

1. **Ràng buộc ô (Cell Constraints)**: Mỗi ô (r, c) phải nhận *đúng một* chữ số từ 1 đến N:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 1 \le d \le N\}\Big), \quad \forall 0 \le r, c < N \tag{3}$$

2. **Ràng buộc hàng (Row Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trên mỗi hàng r:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 0 \le c < N\}\Big), \quad \forall 0 \le r < N,\ 1 \le d \le N \tag{4}$$

3. **Ràng buộc cột (Column Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trên mỗi cột c:

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid 0 \le r < N\}\Big), \quad \forall 0 \le c < N,\ 1 \le d \le N \tag{5}$$

4. **Ràng buộc khối con (Block/Box Constraints)**: Mỗi chữ số d phải xuất hiện *đúng một* lần trong mỗi khối (br, bc):

$$\text{Exactly-One}\Big(\{x_{r,c,d} \mid r \in \text{block}_r(br), c \in \text{block}_c(bc)\}\Big) \tag{6}$$

5. **Ràng buộc gợi ý ban đầu (Clue / Given Constraints)**: Nếu ô (r, c) đã có giá trị cho trước $v \in \{1, \dots, N\}$, mệnh đề đơn nguyên (unit clause) được bổ sung:

$$(x_{r,c,v}) \quad \forall (r, c, v) \in \text{Clues} \tag{7}$$

Một ràng buộc Exactly-One (EO) trên tập biến $X = \{x_1, \dots, x_n\}$ được phân rã thành:

$$\text{EO}(X) \iff \text{At-Least-One}(X) \land \text{At-Most-One}(X) \tag{8}$$

Ràng buộc **At-Least-One (ALO)** được biểu diễn bằng một mệnh đề tuyển:

$$\text{ALO}(X) = (x_1 \lor x_2 \lor \dots \lor x_n) \tag{9}$$

Trong khi đó, việc mã hóa ràng buộc **At-Most-One (AMO)** có ảnh hưởng quyết định đến quy mô của file CNF và thời gian giải của SAT Solver.

3. Các kỹ thuật mã hóa At-Most-One (AMO Encodings)

Cho $X = \{x_1, \dots, x_n\}$, AMO yêu cầu $\sum_{i=1}^{n} x_i \le 1$. Năm kỹ thuật dưới đây được tích hợp trong hệ thống. Các công thức đếm áp dụng cho $n \ge 2$ theo cấu trúc được mô tả; trường hợp nhỏ có thể được thư viện rút gọn.

3.1. Pairwise Encoding (Naive Approach)

Phương pháp tổ hợp cặp đôi cổ điển phát biểu rằng hai biến bất kỳ trong X không thể đồng thời nhận giá trị True:

$$\text{AMO}_{\text{pair}}(X) = \bigwedge_{1 \le i < j \le n} (\neg x_i \lor \neg x_j) \tag{10}$$

- **Số biến phụ**: 0.
- **Số mệnh đề**: $\binom{n}{2} = \frac{n(n-1)}{2} = O(n^2)$.
- **Đặc điểm**: Dễ cài đặt, không làm tăng không gian biến tìm kiếm nhưng số mệnh đề bùng nổ theo hàm bậc hai khi n tăng lớn.

3.2. Binary Encoding (Bitwise Approach)

Phương pháp sử dụng $m = \lceil \log_2 n \rceil$ biến phụ $b_1, \dots, b_m$ để đánh số nhị phân các biến đầu vào. Biến x_i tương ứng với mã của $i - 1$. Khi $x_i = 1$, các bit phụ phải nhận đúng mã này:

$$\text{AMO}_{\text{bin}}(X) = \bigwedge_{i=1}^{n} \bigwedge_{j=1}^{m} \Big(\neg x_i \lor \ell(i,j) \Big) \tag{11}$$

trong đó $\ell(i,j) = b_j$ nếu bit thứ j của $(i-1)$ là 1, và $\ell(i,j) = \neg b_j$ nếu bit thứ j là 0.

- **Số biến phụ**: $\lceil \log_2 n \rceil = O(\log n)$.
- **Số mệnh đề**: $n\lceil \log_2 n \rceil = O(n \log n)$.
- **Đặc điểm**: Tiết kiệm biến phụ và mệnh đề đáng kể khi n rất lớn.

3.3. Commander Variable Encoding

Kỹ thuật do Klieber and Kwon (2007) đề xuất. Ý tưởng chính là phân hoạch n biến thành các nhóm con kích thước k (thường chọn $k = 3$). Mỗi nhóm được chỉ định một biến chỉ huy (*commander variable*) c_g:

1. Bên trong mỗi nhóm con, áp dụng Pairwise AMO: $\bigwedge_{u<v}(\neg u \lor \neg v)$.
2. Ràng buộc biến phần tử với biến chỉ huy: $x \to c_g$ $(\neg x \lor c_g)$.
3. Ràng buộc biến chỉ huy với nhóm: $c_g \to \bigvee_{x \in g} x$ $(\neg c_g \lor \bigvee x)$.
4. Áp dụng đệ quy ràng buộc AMO trên tập biến chỉ huy $\{c_1, c_2, \dots\}$.

- **Số biến phụ**: $O(n/k) = O(n)$.
- **Số mệnh đề**: $O(n)$ (tuyến tính cả về biến và mệnh đề).

3.4. 2-Product Encoding (Jingchao Chen, 2010)

Phương pháp 2-Product được Chen (2010) đề xuất, bố trí n biến vào một lưới 2 chiều kích thước $p \times q$ sao cho $p \times q \geq n$, với $p = \lceil \sqrt{n} \rceil$ và $q = \lceil n/p \rceil$. Ta giới thiệu p biến hàng $r_1, \ldots, r_p$ và q biến cột $c_1, \ldots, c_q$. Mỗi biến $x_{i,j}$ tương ứng với ô ở hàng i, cột j:

$$x_{i,j} \to r_i \iff (\neg x_{i,j} \vee r_i) \tag{12}$$

$$x_{i,j} \to c_j \iff (\neg x_{i,j} \vee c_j) \tag{13}$$

$$\text{AMO}(r_1, \ldots, r_p) \wedge \text{AMO}(c_1, \ldots, c_q) \tag{14}$$

- **Số biến phụ**: $p + q = O(\sqrt{n})$; bằng $2\sqrt{n}$ khi n là số chính phương, không bằng $2\lceil \sqrt{n} \rceil$ trong mọi trường hợp.
- **Số mệnh đề**: $2n + \binom{p}{2} + \binom{q}{2} = O(n)$ khi dùng Pairwise trên các biến hàng và cột như cài đặt hiện tại.
- **Đặc điểm**: Cấu trúc lưới giảm số mệnh đề khi n đủ lớn nhưng phát sinh chi phí bổ sung ở nhóm nhỏ.

3.5. Sequential Counter Encoding

Được Sinz (2005) đề xuất, sử dụng một chuỗi biến đếm $s_1, s_2, \ldots, s_{n-1}$ đại diện cho trạng thái tích lũy của chuỗi tiền tố:

$$(\neg x_1 \vee s_1) \tag{15}$$

$$(\neg x_i \vee s_i), \quad (\neg s_{i-1} \vee s_i), \quad (\neg x_i \vee \neg s_{i-1}) \quad \forall 2 \leq i \leq n-1 \tag{16}$$

$$(\neg x_n \vee \neg s_{n-1}) \tag{17}$$

- **Số biến phụ**: $n - 1$.
- **Số mệnh đề**: $3n - 4 = O(n)$.
- **Lan truyền đơn vị**: Khi một biến đầu vào đúng, chuỗi biến phụ cho phép suy ra các biến đầu vào khác sai mà không cần phân nhánh thêm để thực hiện suy diễn này.

Trong bảng dưới, lan truyền được xét trên biến đầu vào của AMO: một biến đúng suy ra các biến còn lại sai, hai biến đúng dẫn đến xung đột. Binary cũng có tính chất này vì hai mã khác nhau có ít nhất một bit khác nhau. Tính chất này không có nghĩa mọi encoding có chi phí lan truyền hoặc tìm kiếm như nhau.

Bảng 1
So sánh đặc tính lý thuyết của 5 phương pháp mã hóa At-Most-One (AMO).

Phương pháp AMO	Tác giả / Nguồn	Số biến phụ	Số mệnh đề	Đặc tính Unit Propagation
Pairwise (Naive)	Cổ điển	0	$\frac{n(n-1)}{2} = O(n^2)$	Suy diễn trực tiếp
Binary (Bitwise)	Frisch et al. (2008)	$\lceil \log_2 n \rceil$	$n\lceil \log_2 n \rceil = O(n \log n)$	Suy diễn qua các bit phụ
Commander	Klieber and Kwon (2007)	$O(n)$, k cố định	$O(n)$	Suy diễn trong và giữa nhóm
2-Product	Chen (2010)	$p + q = O(\sqrt{n})$	$2n + \binom{p}{2} + \binom{q}{2} = O(n)$	Qua biến hàng và cột
Sequential Counter	Sinz (2005)	$n - 1$	$3n - 4 = O(n)$	Qua chuỗi biến đếm

4. Các phương pháp giải đối chuẩn khác (Alternative Approaches)

Để đối chiếu với SAT, hệ thống mô hình hóa Sudoku trên năm cấu hình bộ giải CP và ILP/MIP:

4.1. Google OR-Tools CP-SAT Solver

CP-SAT là bộ giải Constraint Programming kết hợp SAT hiện đại phát triển bởi Google Perron and Furnon (2024). Trong CP-SAT, Sudoku được mô hình hóa tự nhiên bằng mảng 2 chiều các biến nguyên $y_{r,c} \in \{1, 2, \dots, N\}$. Các ràng buộc hàng, cột và khối con được biểu diễn thông qua ràng buộc toàn cục:

$$\texttt{AddAllDifferent}\Big(\{y_{r,c} \mid c = 0 \dots N-1\}\Big) \tag{18}$$

Mã nguồn khai báo `AddAllDifferent` cho hàng, cột và khối, cùng đẳng thức cho gợi ý. Việc tiền xử lý và lựa chọn cơ chế lan truyền do OR-Tools quyết định; báo cáo không giả định một thuật toán lọc miền cụ thể luôn được kích hoạt.

4.2. IBM ILOG CPLEX CP Optimizer

Bộ giải CP của IBM sử dụng biến `integer_var(1, N)`, ràng buộc `all_diff()` cho hàng, cột, khối và đẳng thức cho gợi ý. Hệ thống gọi bộ máy của CP Optimizer, không tự cài đặt hoặc cấu hình chiến lược tìm kiếm cụ thể.

4.3. Integer Linear Programming (ILP/MIP): Gurobi, CPLEX MIP & PuLP/CBC

Trong mô hình Quy hoạch tuyến tính nguyên:

- Biến quyết định nhị phân: $z_{r,c,d} \in \{0, 1\}$.
- Hàm mục tiêu: Tìm nghiệm thỏa mãn ràng buộc (Hàm mục tiêu bằng 0 hoặc hằng số): min 0.
- Hệ thống ràng buộc đẳng thức tuyến tính:

$$\sum_{d=1}^{N} z_{r,c,d} = 1 \quad \forall r, c \tag{19}$$

$$\sum_{c=0}^{N-1} z_{r,c,d} = 1 \quad \forall r, d \tag{20}$$

$$\sum_{r=0}^{N-1} z_{r,c,d} = 1 \quad \forall c, d \tag{21}$$

$$\sum_{r \in \text{block}} \sum_{c \in \text{block}} z_{r,c,d} = 1 \quad \forall \text{block}, d \tag{22}$$

Mô hình được giải bằng Gurobi, CPLEX MIP hoặc CBC thông qua PuLP. Gợi ý được cố định bằng $z_{r,c,v} = 1$. Đây là bài toán tìm nghiệm khả thi, không phải tối ưu chất lượng lời giải Sudoku.

5. Thiết kế và cài đặt hệ thống

5.1. Kiến trúc và luồng xử lý

Chương trình được tổ chức thành các mô-đun độc lập: `sat_encodings/` sinh ALO, AMO và EO; `solvers/` xây dựng mô hình và gọi bộ giải; `benchmarks/` nạp dữ liệu, đo thời gian và vẽ biểu đồ; `tests/` kiểm thử; `main.py` điều phối giao diện dòng lệnh; `utils.py` cung cấp bảng mẫu, suy ra kích thước khối và in bảng.

Luồng benchmark gồm: đọc snapshot và lọc đề; xây dựng CNF hoặc mô hình CP/MIP; gọi solver; giải mã kết quả; kiểm tra hàng, cột, khối và việc giữ nguyên gợi ý; tổng hợp thống kê rồi xuất kết quả. SAT sử dụng N^3 biến cơ sở và cấp các ID phụ phía sau để tránh trùng biến. Pairwise, Commander và Product tự sinh mệnh đề bằng Python; Binary và Sequential Counter gọi `CardEnc.atmost` của PySAT Ignatiev et al. (2018) với kiểu mã hóa tương ứng.

5.2. Chức năng và cách sử dụng

CLI hỗ trợ kích thước 4, 9 và 16; giải một bảng mẫu bằng SAT; benchmark nhanh trên bảng mẫu hoặc benchmark tập dữ liệu; lọc theo kích thước, nguồn và nhãn độ khó; giới hạn số bản ghi; xuất CSV/JSON, biểu đồ và tên tệp có dấu thời gian. API Python cho phép truyền ma trận đề trực tiếp, còn chế độ giải đơn lẻ của CLI hiện chỉ chọn bảng mẫu và encoding, chưa có chức năng nhập đề tùy ý qua tệp.

```
pip install -r requirements.txt
python main.py --size 9 --solve product
python main.py --benchmark-dataset-encodings --size 9 --plot --csv --json
python main.py --benchmark-dataset-solvers --size 9 --plot --csv --json
python main.py --test
```

Các lệnh trên minh họa cách sử dụng. Python từ 3.10 trở lên được yêu cầu theo tài liệu dự án. Gurobi và IBM CPLEX cần thư viện và quyền sử dụng phù hợp; lỗi license hoặc môi trường phải được phân biệt với kết quả không có nghiệm. Chức năng `--prepare-data` tải lại nguồn động, không bảo đảm tái tạo đúng snapshot 299 bản ghi đang đóng gói.

5.3. Kiểm thử và kiểm tra nghiệm

`test_encodings.py` duyệt 2^6 phép gán cho ALO, từng AMO và EO với năm phương pháp, đối chiếu tính khả thỏa với điều kiện số biến đúng. Đây là kiểm thử hữu hạn ở $n = 6$, chưa bao phủ mọi kích thước hoặc các trường hợp rỗng và đơn phần tử.

`test_solvers.py` kiểm tra bảng mẫu 9×9 với các solver; ở 4×4 kiểm tra SAT Product, CP-SAT và CBC; ở 16×16 kiểm tra SAT Product và CP-SAT. `test_dataset.py` kiểm tra chuyển đổi chuỗi–ma trận, định dạng dữ liệu và suy ra độ khó. Bộ kiểm tra nghiệm đối chiếu giá trị, hàng, cột, khối và gợi ý, không chứng minh nghiệm duy nhất. Các bài kiểm tra solver thương mại có cơ chế bỏ qua khi lỗi môi trường; do đó cần xem log cụ thể, không suy ra mọi solver đã vượt qua kiểm thử chỉ từ việc chương trình kết thúc. Báo cáo mô tả phạm vi mã kiểm thử hiện có, không công bố một lần chạy kiểm thử mới.

6. Kết quả thực nghiệm và Thảo luận (Experimental Evaluation)

6.1. Môi trường thực nghiệm & Bộ dữ liệu Benchmark

- **Môi trường phần mềm**: Python 3.14, thư viện PySAT (python-sat 1.9), OR-Tools 9.15, Gurobi 13.0, CPLEX 22.2 và PuLP 3.3.
- **Dữ liệu Benchmark**: 299 bản ghi trong `data/benchmark_dataset.json`:
  - Bảng 9×9: 279 bản ghi, gồm *19 Easy, 77 Medium, 99 Hard, 84 Expert*; dùng cho Bảng 3 và Hình 1–3.
  - Bảng 4×4: 10 đề tự sinh.
  - Bảng 16×16: 10 đề tự sinh, tỷ lệ ô giữ lại được cấu hình từ 18% đến 55%. Bảng 2 trình bày cả nhóm 9×9 và 16×16.

6.2. Nguồn, chuẩn hóa và đặc điểm dữ liệu

Snapshot có 131 bản ghi Norvig (70 top95, 50 easy50, 11 hardest), 148 bản ghi Sudoku Exchange (43 easy, mỗi nhóm medium/hard/diabolical có 35), và 20 bản ghi tự sinh. Các nguồn trực tuyến được tham khảo tại `https://norvig.com/sudoku.html` và `https://github.com/grantm/sudoku-exchange-puzzle-bank`.

Schema thống nhất gồm `id`, `source`, `dataset`, `size`, `difficulty`, `clues_count`, `puzzle_string`, `grid` và các trường nghiệm tùy chọn. Ô trống có giá trị 0; chuỗi dùng ký hiệu chữ cho các giá trị lớn hơn 9. Snapshot hiện chưa có đáp án chuẩn trong trường `solution_grid`; tính hợp lệ được kiểm tra bằng luật Sudoku thay vì đối chiếu đáp án lưu sẵn.

Loader ưu tiên nhãn nguồn khi nhận diện được; nếu không, suy ra theo mật độ $\rho = g/N^2$: Expert khi $\rho \leq 0.27$, Hard khi $0.27 < \rho \leq 0.33$, Medium khi $0.33 < \rho \leq 0.42$, Easy khi $\rho > 0.42$. Nhãn của snapshot được dùng nguyên trạng khi phân tích; một số nhãn khác với quy tắc ưu tiên nguồn của loader hiện tại nên không được coi là một chuẩn độ khó thống nhất giữa các nguồn. Số gợi ý chỉ là chỉ báo, không xác định đầy đủ độ khó đối với solver.

Hai bản ghi `norvig_easy50_0009` và `norvig_easy50_0041` trùng đề; do đó có 298 đề phân biệt (278 đề 9×9), dù các thống kê cũ vẫn tính trên 299 bản ghi. Các đề 16×16 được sinh bằng cách xóa ô từ một nghiệm cơ sở với bộ sinh ngẫu nhiên seed 42. Cách này bảo đảm tồn tại ít nhất một nghiệm nhưng chưa bảo đảm nghiệm duy nhất hoặc độ đa dạng tương đương dữ liệu thu thập độc lập.

6.3. Định nghĩa phép đo và phạm vi số liệu

Benchmark encoding đo riêng thời gian sinh CNF và thời gian gọi `solver.solve()`; tổng là tổng hai pha này, không gồm khởi tạo solver, nạp CNF, giải mã hoặc kiểm tra nghiệm. Benchmark solver đo toàn bộ lời gọi hàm giải, bao gồm dựng mô hình và các chi phí nằm trong wrapper, nhưng không gồm bước kiểm tra nghiệm bên ngoài. Vì vậy hai loại thời gian không thể so sánh trực tiếp.

Các bảng và hình 9×9 cùng thời gian case study trình bày số liệu đã ghi nhận nhưng chưa có đủ dữ liệu thô để xác minh độc lập toàn bộ kết quả. Riêng thực nghiệm bổ sung 16×16 ở Mục 6.7 có CSV/JSON chi tiết và thông tin môi trường đi kèm. Hai nhóm kết quả được phân biệt về nguồn gốc; lượt đo mới không được dùng để xác nhận lại các thời gian cũ. Quy mô CNF có thể đối chiếu bằng công thức và đầu vào.

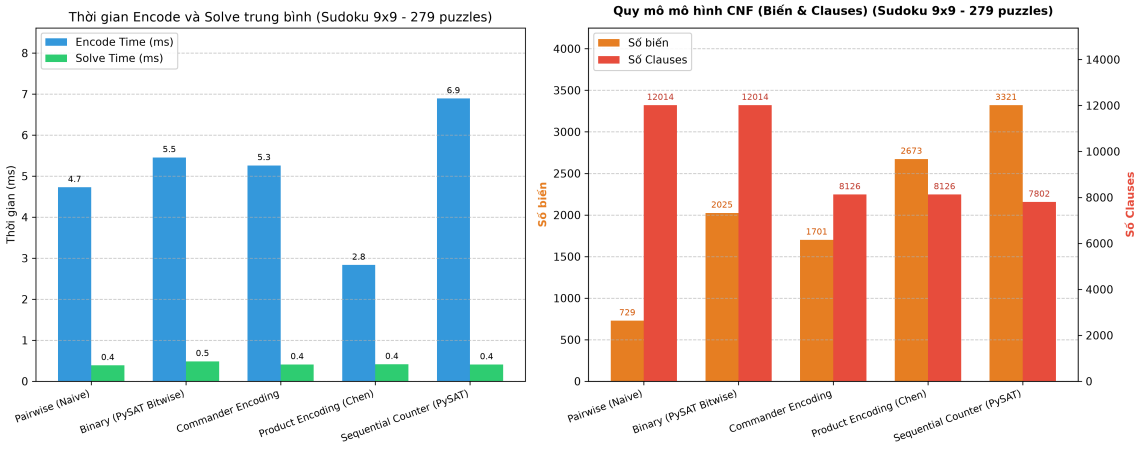
Các thời gian 9×9 và case study được sử dụng như số liệu khảo sát, không phải bằng chứng về thứ hạng ổn định. Báo cáo không có đủ dữ liệu đo lặp để tính khoảng tin cậy hoặc kiểm định ý nghĩa của các chênh lệch nhỏ; việc trình bày hai chữ số thập phân chỉ là quy ước làm tròn, không hàm ý độ chính xác tương ứng của phép đo.

6.4. Đánh giá quy mô mô hình SAT Encodings (Variables và Clauses)

Bảng 2 trình bày hai nhóm kích thước; Hình 1 chỉ dùng nhóm 9×9. Với g gợi ý và $a(N)$ mệnh đề AMO mỗi nhóm, tổng CNF là $4N^2[1 + a(N)] + g$. Số mệnh đề trung bình được làm tròn đến số nguyên.

Bảng 2
Quy mô trung bình số biến và mệnh đề sinh ra giữa 5 phương pháp AMO trên Sudoku 9×9 (279 câu đố) và 16×16 (10 câu đố).

Phương pháp AMO	Sudoku 9x9		Sudoku 16x16	
	Số biến	Số Clauses	Số biến	Số Clauses
Pairwise (Naive)	729	12,014	4,096	123,987
Binary (Bitwise)	2,025	12,014	8,192	66,643
Commander	1,701	8,126	11,264	52,307
2-Product	2,673	8,126	12,288	46,163
Sequential Counter	3,321	7,802	19,456	46,163



Hình 1: Thời gian mã hóa, gọi solve() và quy mô CNF trên 279 bản ghi 9×9; tổng hai pha không gồm khởi tạo và nạp CNF.

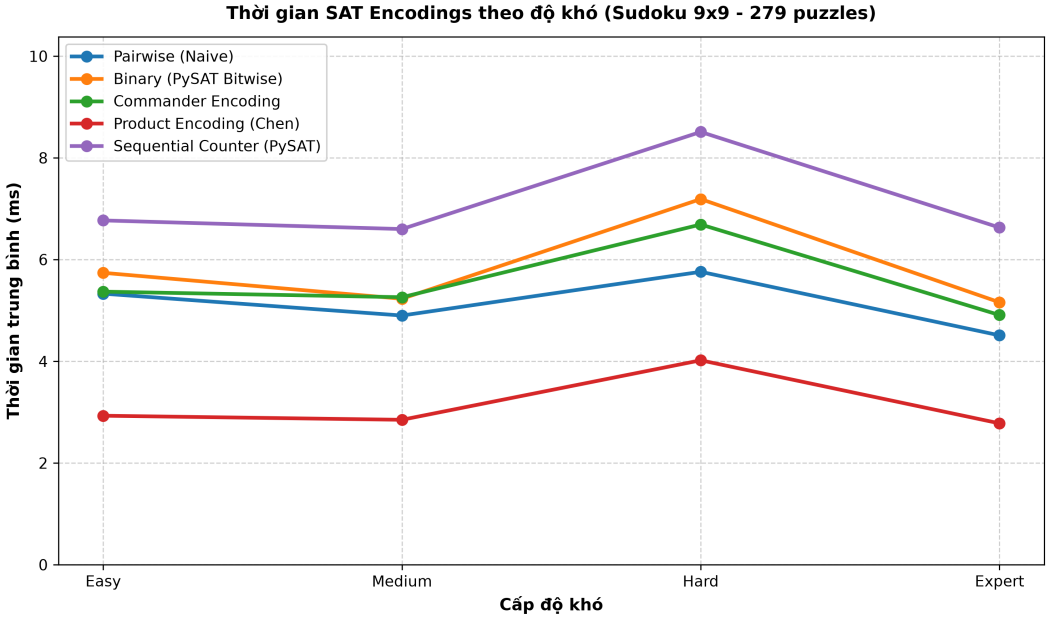
Phân tích:

- *Về số mệnh đề*: Pairwise sinh khoảng 12,014 mệnh đề ở 9×9 và 123,987 ở 16×16. Ở $N = 9$, Binary bằng Pairwise vì $\binom{9}{2} = 36 = 9\lceil \log_2 9 \rceil$; ở $N = 16$, Binary giảm khoảng 46%. Commander, Product và Sequential Counter giảm khoảng 32–35% ở 9×9 và 58–63% ở 16×16. Tính trên toàn bảng, Pairwise có quy mô $O(N^4)$, Binary là $O(N^3 \log N)$ và các AMO tuyến tính cho quy mô $O(N^3)$.

- *Về thời gian*: Trong số liệu nhóm 9 × 9, mã hóa mất khoảng 2.8–6.9 ms, còn gọi giải mất 0.4–0.5 ms. Product cho kết quả tốt ở phép đo hai pha. Chưa thể quy chênh lệch cho riêng unit propagation hoặc độ phức tạp tiệm cận vì cách cài đặt và các chi phí ngoài timer cũng ảnh hưởng đến thực thi.

6.5. Phân tích độ nhạy cảm theo cấp độ khó (Difficulty Analysis)

Hình 2 biểu diễn tổng thời gian (mã hóa + giải) trung bình của 5 phương pháp SAT Encoding qua 4 cấp độ khó: *Easy → Medium → Hard → Expert*.



Hình 2: Xu hướng biến thiên tổng thời gian (mã hóa + giải) của 5 kỹ thuật SAT Encoding theo các cấp độ khó (Sudoku 9x9).

Nhận xét:

- **Khác biệt giữa các encoding**: Product có tổng thời gian mã hóa và gọi giải tương đối thấp ở cả bốn nhóm, khoảng 2.8–4.1 ms. Sequential Counter nằm ở mức cao hơn, khoảng 6.5–8.6 ms, dù sinh ít mệnh đề. Quan sát này gợi ý rằng số mệnh đề chỉ phản ánh một phần chi phí; cách sinh CNF và số biến phụ cũng là những yếu tố đáng xem xét.

- **Xu hướng theo nhãn độ khó**: Thời gian trung bình không tăng đều từ Easy đến Expert; trong hình, nhóm Hard có giá trị cao hơn nhóm Expert. Vì nhãn được tổng hợp từ nhiều nguồn và một phần dựa trên mật độ gợi ý, thứ tự các nhãn không nhất thiết tương ứng với mức độ khó đối với Glucose3.

- **Vai trò của hai pha**: Trên nhóm bảng 9 × 9, thời gian mã hóa chiếm phần lớn tổng hai pha, trong khi thời gian gọi giải trung bình ở mức khoảng 0.4–0.5 ms. Vì vậy, khác biệt về quá trình tìm nghiệm có thể chưa thể hiện rõ trên đường tổng thời gian. Việc xem riêng pha giải sẽ giúp bổ sung góc nhìn về ảnh hưởng của cấu trúc từng đề.

- Nhìn chung, hình cho thấy các encoding có mức chi phí khác nhau nhưng chưa thể hiện một quan hệ đơn giản giữa nhãn độ khó và thời gian xử lý. Các ví dụ ở Mục 7 được dùng để quan sát thêm sự khác biệt trên từng đề, thay vì đại diện cho toàn bộ một cấp độ.

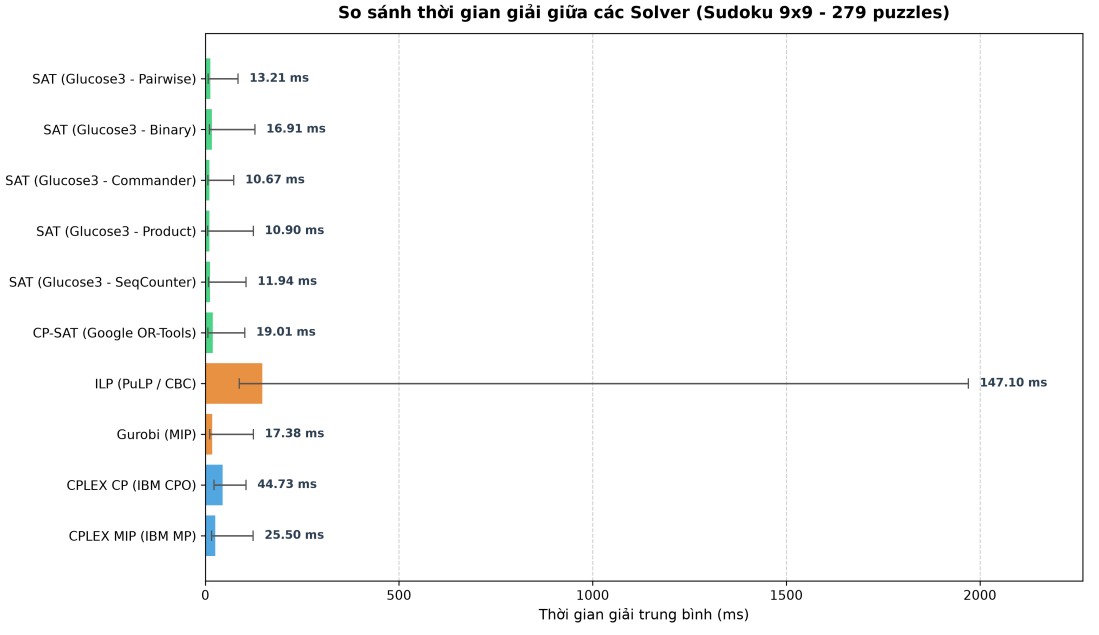
6.6. So sánh đối chuẩn hiệu năng giữa SAT Solver và các Solvers (CP, ILP/MIP)

Bảng 3 và Hình 3 minh họa kết quả đo lường thời gian thực thi trung bình giữa các công cụ giải khác nhau trên cùng tập dữ liệu chuẩn hóa.

Thảo luận:

Bảng 3
Thời gian toàn bộ hàm giải trên 279 bản ghi 9×9; tỷ lệ nghiệm hợp lệ theo số liệu đã ghi nhận.

Bộ giải (Solver Engine)	Trường phái	Thời gian TB (ms)	Tỷ lệ thành công
SAT (Glucose3 - Commander)	SAT	**10.67**	100%
SAT (Glucose3 - Product)	SAT	**10.90**	100%
SAT (Glucose3 - SeqCounter)	SAT	**11.94**	100%
SAT (Glucose3 - Pairwise)	SAT	**13.21**	100%
SAT (Glucose3 - Binary)	SAT	**16.91**	100%
Gurobi Optimizer	MIP	17.38	100%
Google OR-Tools CP-SAT	CP / SAT Hybrid	19.01	100%
IBM CPLEX MIP	MIP	25.50	100%
IBM CPLEX CP Optimizer	CP	44.73	100%
ILP (PuLP / CBC)	ILP / Branch-and-Cut	147.10	100%



Hình 3: Thời gian thực thi các hàm giải trên 279 bản ghi Sudoku 9×9.

1. **Các cấu hình SAT**: Thời gian trung bình nằm trong khoảng 10.67–16.91 ms. Commander và Product có kết quả khá gần nhau (10.67 và 10.90 ms), nên có thể xem chúng là hai lựa chọn đáng chú ý trong phép đo này, thay vì nhấn mạnh thứ hạng từ một chênh lệch nhỏ.
2. **Đối chiếu với CP và MIP**: Gurobi (17.38 ms) và CP-SAT (19.01 ms) có thời gian cùng bậc độ lớn với nhóm SAT; Gurobi khá gần SAT Binary (16.91 ms). CPLEX MIP, CP Optimizer và CBC có thời gian cao hơn trong bảng, nhưng đều được ghi nhận tìm được nghiệm hợp lệ trên tập khảo sát. Kết quả cho thấy cả ba hướng mô hình hóa đều sử dụng được cho các đề 9×9 này, với chi phí thực thi khác nhau.
3. **Liên hệ với phép đo mã hóa**: Product có kết quả thuận lợi khi xét riêng mã hóa cộng gọi giải ở Mục 6.4, còn Commander có giá trị trung bình thấp hơn một chút khi đo toàn bộ hàm giải. Hai quan sát này không mâu thuẫn: phép đo toàn bộ còn tính khởi tạo bộ giải, nạp CNF và giải mã. Việc chọn mã hóa vì thế nên gắn với công đoạn hoặc mục tiêu sử dụng cần quan tâm.
4. **Cách đọc chênh lệch thời gian**: CBC có khoảng cách rõ hơn so với các cấu hình còn lại, trong khi một số cặp solver chỉ chênh vài mili-giây. Các giá trị phản ánh đồng thời cách dựng mô hình, lớp giao tiếp Python và bộ máy giải, nên phù hợp để so sánh các cài đặt trong dự án hơn là đánh giá riêng thuật toán bên trong từng solver. Đo lặp và tách thời gian dựng mô hình khỏi tìm kiếm sẽ giúp làm rõ hơn những khác biệt này.

6.7. So sánh các bộ giải trên Sudoku 16×16

Để bổ sung đánh giá ở kích thước lớn hơn, hệ thống được chạy trên toàn bộ 10 đề 16×16 trong snapshot. Mỗi cặp đề–cấu hình được đo một lần theo thứ tự cố định, tính toàn bộ hàm giải và kiểm tra nghiệm bên ngoài timer. Các gói chính là python-sat 1.9.dev15, OR-Tools 9.15.6755, PuLP 3.3.2, Gurobi 13.0.3, CPLEX 22.2.0.1 và DOcplex 2.32.264. Mã nguồn tương ứng; cấu hình đầy đủ, checksum dữ liệu và kết quả từng đề được lưu trong `reports/submission_16x16/`. Lượt đo này chưa warm-up hoặc đảo thứ tự cấu hình; số luồng, seed và timeout chưa được chuẩn hóa chung giữa các bộ giải. Vì vậy, thời gian phản ánh cả mô hình, cách cài đặt và cấu hình mặc định của từng công cụ, không tách riêng hiệu quả thuật toán bên trong bộ giải.

Bảng 4
Một lượt đo trên 10 đề Sudoku 16×16. Min–max là khoảng thời gian giữa các đề khác nhau, không phải biến thiên qua nhiều lần đo cùng đề. N/A: không có thời gian giải do giới hạn license.

Cấu hình	TB (ms)	Min–max (ms)	Kết quả
SAT Pairwise	108.17	66.17–159.95	10/10 nghiệm hợp lệ
SAT Binary	77.05	43.63–115.79	10/10 nghiệm hợp lệ
SAT Commander	56.79	35.61–111.68	10/10 nghiệm hợp lệ
SAT Product	52.85	27.95–122.33	10/10 nghiệm hợp lệ
SAT Sequential Counter	65.02	36.69–111.30	10/10 nghiệm hợp lệ
OR-Tools CP-SAT	139.57	21.66–276.75	10/10 nghiệm hợp lệ
PuLP/CBC	934.46	235.95–1434.56	10/10 nghiệm hợp lệ
Gurobi MIP	N/A	N/A	10/10 lỗi license
CPLEX CP Optimizer	61.24	42.43–98.52	10/10 nghiệm hợp lệ
CPLEX MIP	N/A	N/A	10/10 lỗi license

Nhận xét:

- Năm encoding SAT, CP-SAT, CBC và CP Optimizer đều trả nghiệm hợp lệ cho cả 10 đề. Product và Commander có thời gian trung bình tương đối thấp trong lượt đo; CP Optimizer cũng nằm gần nhóm này. Các khoảng min–max cho thấy chi phí khác nhau giữa các đề, nên trung bình chỉ tóm tắt một phần kết quả.
- CP-SAT và CBC có khoảng thời gian rộng hơn trong tập khảo sát. Ngoài cấu trúc đề, quá trình dựng mô hình và giao tiếp solver có thể góp phần vào khác biệt; phép đo hiện chưa tách các thành phần này. Do mỗi cấu hình chỉ chạy một lần trên từng đề, thứ hạng được xem là quan sát của lượt chạy thay vì kết luận ổn định.
- Gurobi báo mô hình vượt giới hạn kích thước license. CPLEX MIP bản promotional giới hạn 1.000 biến và 1.000 ràng buộc, trong khi mô hình 16×16 có 4.096 biến và vượt giới hạn ràng buộc. Hai trường hợp được ghi N/A, không diễn giải thành đề vô nghiệm hay mô hình sai.
- Tập 16×16 gồm đề tự sinh từ một nghiệm cơ sở, chưa đại diện cho mọi Hexadoku. Kết quả bổ sung bằng chứng chương trình xử lý được bảng lớn hơn, đồng thời cho thấy cần mở rộng dữ liệu và đo lặp nếu muốn phân tích hiệu năng sâu hơn.

7. Nghiên cứu trường hợp thực nghiệm (Experimental Case Studies)

Các bảng và hình tổng hợp dùng tập con ghi tại chú thích, không phải tất cả đều lấy trung bình trên 299 bản ghi. Mục này giữ lại các số liệu từng đề để minh họa kích thước, nhãn độ khó và encoding.

Phạm vi kiểm chứng. Các đề được định danh cụ thể trong từng bảng. Thời gian case study là số liệu tham khảo đã ghi nhận; bộ mã và dữ liệu thô đi kèm chưa đủ để tái tạo việc chọn mẫu theo trung vị gợi ý hoặc xác nhận giao thức lấy cực tiểu 60 lần trong ba phiên. Do đó, các bảng không được dùng như ước lượng thời gian điển hình hay bằng chứng về độ ổn định. Số biến và mệnh đề có thể đối chiếu với đầu vào và công thức. Lấy cực tiểu nhiều lần cũng không bảo đảm loại bỏ hoàn toàn nhiễu.

7.1. Trường hợp theo kích thước bảng (4×4, 9×9, 16×16)

Ba câu đố được chọn để minh họa lần lượt là `matrix4x4_0002` (4×4, 4 gợi ý), `norvig_top95_0053` (9×9, 26 gợi ý) và `hexadoku16x16_0005` (16×16, 77 gợi ý). Bảng 5 tổng hợp quy mô CNF và thời gian tương ứng; các đề này không được xem là mẫu đại diện thống kê cho từng kích thước.

Bảng 5
Quy mô CNF và thời gian tham khảo đã ghi nhận trên ba đề. Seq. Counter là Sequential Counter; tổng chỉ gồm mã hóa và gọi solve().

Bảng	Puzzle	Gợi ý	Mã hóa AMO	Biến	Mệnh đề	Enc. (ms)	Solve (ms)	Tổng (ms)
4×4	`matrix4x4_0002`	4	Pairwise	64	452	0.17	0.05	0.22
			Binary	192	580	0.58	0.07	0.65
			Commander	128	580	0.34	0.07	0.41
			2-Product	320	708	0.33	0.08	0.41
			Seq. Counter	256	580	0.73	0.06	0.79
9×9	`norvig_top95_0053`	26	Pairwise	729	12,014	2.80	0.45	3.25
			Binary	2,025	12,014	6.81	0.53	7.34
			Commander	1,701	8,126	3.05	0.43	3.48
			2-Product	2,673	8,126	2.50	0.43	**2.93**
			Seq. Counter	3,321	7,802	5.50	0.38	5.88
16×16	`hexadoku16x16_0005`	77	Pairwise	4,096	123,981	30.12	4.10	34.22
			Binary	8,192	66,637	30.14	2.78	32.92
			Commander	11,264	52,301	20.51	3.26	23.77
			2-Product	12,288	46,157	15.63	2.24	**17.87**
			Seq. Counter	19,456	46,157	30.73	3.53	34.26

Nhận xét:

- Pairwise sinh 123, 981 mệnh đề trên đề 16×16, gấp khoảng 2.69 lần Product hoặc Sequential Counter (46, 157). AMO Pairwise tăng $O(n^2)$ trong một nhóm; vì Sudoku có $4N^2$ nhóm nên tổng CNF tăng $O(N^4)$, so với $O(N^3)$ khi dùng AMO tuyến tính.
- Product có tổng hai pha thấp nhất trên hai đề 9×9 và 16×16 trong bảng. Một đề mỗi kích thước chưa đủ đại diện hiệu năng toàn nhóm.
- Với 4×4, Product sinh 10 mệnh đề AMO mỗi nhóm so với 6 của Pairwise; cộng ALO thì có 11 và 7 mệnh đề EO. Tổng lần lượt là $64 \times 11 + 4 = 708$ và $64 \times 7 + 4 = 452$. Ưu thế tiệm cận không bảo đảm ưu thế ở nhóm nhỏ.
- Mã hóa lâu hơn gọi giải ở các hàng trong bảng, nhưng tỷ trọng này không gồm nạp CNF và không thay thế phép đo toàn bộ hàm giải.

7.2. Trường hợp theo cấp độ khó

Hai bảng 6 và 7 trình bày kết quả minh họa trên bốn câu đố 9×9 mang nhãn Easy, Medium, Hard và Expert, với số gợi ý giảm từ 36 xuống 23. Mỗi nhãn chỉ có một đề trong case study nên không thể suy ra thời gian điển hình của cả nhóm.

Bảng 6
Tổng thời gian (mã hóa + giải, ms) trên bốn câu đố 9×9 đại diện: `norvig_easy50_0005` (Easy), `norvig_easy50_0003` (Medium), `norvig_top95_0012` (Hard) và `norvig_hardest_0002` (Expert).

Độ khó	Gợi ý	Pairwise	Binary	Commander	2-Product	Seq. Counter
Easy	36	3.23	7.85	3.81	3.27	5.00
Medium	28	3.21	6.20	3.52	3.50	6.44
Hard	25	3.54	7.01	4.03	3.63	7.01
Expert	23	4.13	8.32	5.53	4.90	7.37

Nhận xét:

- Từ đề Easy đến Expert, Product tăng 3.27 $\rightarrow$ 4.90 ms, Sequential Counter 5.00 $\rightarrow$ 7.37 ms và Pairwise 3.23 $\rightarrow$ 4.13 ms. Dãy số không tăng đơn điệu ở mọi phương pháp; Product không thấp nhất trong cả bốn hàng.

Bảng 7
Thời gian giải thuần túy (ms, chỉ tính pha `Glucose3.solve()`) trên cùng bốn câu đố 9×9 đại diện theo cấp độ khó.

Độ khó	Gợi ý	Pairwise	Binary	Commander	2-Product	Seq. Counter
Easy	36	0.32	0.48	0.32	0.33	0.21
Medium	28	0.34	0.40	0.27	0.33	0.26
Hard	25	0.47	0.63	0.66	0.42	0.78
Expert	23	0.67	1.14	1.52	1.60	1.36

- Tỷ số thời gian gọi giải Expert/Easy khoảng 2.1–6.5 lần, riêng Sequential Counter là $1.36/0.21 \approx 6.5$. Bốn đề khác nhau chưa đủ để tách ảnh hưởng số gợi ý khỏi cấu trúc đề và nguồn dữ liệu.
- Các bảng từng đề và Hình 2 dùng tập quan sát khác nhau nên xu hướng không nhất thiết trùng nhau. Chưa có dữ liệu thô để quy khác biệt cho GC, turbo boost hay một ngoại lệ cụ thể. Cần đo lặp có kiểm soát trước khi kết luận độ nhạy theo độ khó.

7.3. Trường hợp theo kỹ thuật mã hóa

Bảng 8 phân rã cấu trúc file CNF của câu đố 9×9 đại diện (26 gợi ý) theo từng kỹ thuật mã hóa.

Bảng 8
Phân rã quy mô CNF của đề `norvig_top95_0053` (9×9, 26 gợi ý). Mọi cấu hình có 729 biến cơ sở, 324 mệnh đề ALO và 26 mệnh đề gợi ý; tổng số biến, mệnh đề và thời gian đã trình bày ở Bảng 5.

Phương pháp	Biến phụ	AMO mỗi nhóm	Tổng AMO
Pairwise	0	36	11,664
Binary	1,296	36	11,664
Commander	972	24	7,776
2-Product	1,944	24	7,776
Sequential Counter	2,592	23	7,452

Nhận xét:

- Có 324 EO, 324 ALO và 26 mệnh đề gợi ý. AMO mỗi nhóm sinh lần lượt 36, 36, 24, 24, 23 mệnh đề, phù hợp các công thức đã trình bày. Đây là kiểm tra quy mô, chưa phải chứng minh đúng đắn ngữ nghĩa.
- Biến phụ lần lượt là $324 \times 4 = 1,296$ (Binary), $324 \times 3 = 972$ (Commander ở kích thước này), $324 \times 6 = 1,944$ (Product), $324 \times 8 = 2,592$ (Sequential Counter). Với Commander lớn hơn cần tính cả các tầng đệ quy.
- Product mã hóa nhanh nhất (2.50 ms), nhưng Sequential Counter gọi giải nhanh nhất (0.38 ms). Khác biệt giữa gọi PySAT và tự sinh Python chưa đủ để kết luận chi phí thư viện là nguyên nhân chính.

Bảng 9 minh họa đề có 26 gợi ý và một nghiệm hợp lệ được ghi nhận với Glucose3, mã hóa Product. Nghiệm thỏa hàng, cột, khối và gợi ý; chưa khẳng định duy nhất vì quy trình hiện chỉ tìm một nghiệm, không loại nghiệm đó để kiểm tra nghiệm thứ hai.

Giới hạn và hướng kiểm chứng tiếp theo

Các giới hạn chính gồm: (i) dữ liệu còn trùng đề, nhãn độ khó chưa đồng nhất và nhóm 16×16 nhỏ, được sinh từ một nghiệm cơ sở; (ii) số liệu thời gian cũ thiếu dữ liệu thô để xác minh đầy đủ, còn lượt đo bổ sung chưa có đo lặp và kiểm soát thống nhất cấu hình bộ giải; (iii) các phép đo chịu ảnh hưởng của dựng mô hình và giao tiếp thư viện; (iv) kiểm tra nghiệm chưa chứng minh tính duy nhất. Do đó, báo cáo không kết luận quan hệ nhân quả giữa số mệnh đề, lan truyền đơn vị và tốc độ, cũng không khẳng định một phương pháp luôn tốt nhất.

Để kiểm chứng sâu hơn, cần làm sạch snapshot, bổ sung đề độc lập, lưu dữ liệu từng lần chạy, warm-up và đảo thứ tự cấu hình; đồng thời công bố số luồng, seed, timeout, trung vị, độ phân tán và phạm vi timer. Các bước này là hướng phát triển, chưa phải thực nghiệm đã thực hiện trong báo cáo.

Bảng 9
Câu đố 9×9 đại diện `norvig_top95_0053` (26 gợi ý) và nghiệm do Glucose3 với mã hóa 2-Product tìm được.

Đề bài

	2					5	9	3
8			5			4	6	
9	4			6				8
		2		3				
	6			8		7	3	
7			2					
				4		3	8	
	7					6		
								5

Nghiệm

1	2	6	4	7	8	5	9	3
8	3	7	5	9	2	4	6	1
9	4	5	3	6	1	2	7	8
4	1	2	9	3	7	8	5	6
5	6	9	1	8	4	7	3	2
7	8	3	2	5	6	9	1	4
2	5	1	6	4	9	3	8	7
3	7	4	8	1	5	6	2	9
6	9	8	7	2	3	1	4	5

8. Kết luận và Hướng phát triển (Conclusion)

Bài tập lớn xây dựng hệ thống Sudoku bằng SAT và tích hợp CP/MIP để so sánh. Pairwise, Commander và Product được tự cài đặt; Binary và Sequential Counter sử dụng PySAT. Công thức CNF thể hiện sự đánh đổi giữa biến phụ và mệnh đề khi tăng kích thước.

Trong số liệu đã ghi nhận, Product có chi phí hai pha thấp ở nhiều cấu hình, còn Commander có thời gian toàn bộ hàm giải trung bình thấp nhất ở bảng 9×9. Thực nghiệm bổ sung trên 10 đề 16×16 cho nghiệm hợp lệ với năm encoding SAT, CP-SAT, CBC và CP Optimizer; Gurobi và CPLEX MIP cần license cho mô hình lớn hơn. Không có encoding tốt nhất độc lập với tiêu chí đo, kích thước và cài đặt. Kết quả chưa chứng minh ưu thế tổng quát của SAT trước CP/MIP.

Hướng phát triển gần nhất là hoàn thiện kiểm tra đầu vào, khử trùng dữ liệu, kiểm tra duy nhất, bổ sung đo lặp và dữ liệu thô. Sau đó có thể khảo sát Diagonal hoặc Killer Sudoku bằng ràng buộc phù hợp; SMT và MaxSAT là hướng bổ sung, không bắt buộc cho mọi biến thể.

Tính sẵn có của mã nguồn và dữ liệu (Data Availability)

Mã nguồn, snapshot 299 bản ghi, script benchmark và biểu đồ được lưu tại dự án dưới đây. Dữ liệu thô của lượt đo 16×16 gồm `solvers.csv`, `solvers.json`, thông tin môi trường `environment.json` và hướng dẫn chạy lại trong `reports/submission_16x16/`. Các tệp giữ cả kết quả hợp lệ lẫn thông báo lỗi license để người đọc đối chiếu Bảng 4.

`https://github.com/nmson8206/Sudoku_prj`

Tài liệu

Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence, pp. 399–404.

Chen, J., 2010. A new SAT encoding of the at-most-one constraint.

Frisch, A.M., Peugniez, N., Doggett, A.J., Nightingale, P., 2008. Effective propositional encodings of constraint satisfaction problems. Artificial Intelligence 172, 1961–1994.

Gurobi Optimization, LLC, 2024. Gurobi optimizer reference manual. Online documentation. URL: `https://www.gurobi.com`.

IBM Corporation, 2024. IBM ILOG CPLEX optimization studio user's manual. Online documentation. URL: `https://www.ibm.com/analytics/cplex-optimizer`.

Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.

Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from n objects.

Norvig, P., 2006. Solving every Sudoku puzzle. Online essay. URL: `https://norvig.com/sudoku.html`.

Perron, L., Furnon, V., 2024. Google OR-Tools: CP-SAT solver reference. Google Developers documentation. URL: `https://developers.google.com/optimization/cp/cp_solver`.

Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831.

Yato, T., Seta, T., 2003. Complexity and completeness of finding another solution and its application to puzzles. IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences E86-A, 1052–1060.